Federated Learning (FL)

Privacy, Security, and Utility Tradeoffs in Federated Wearable Activity Recognition

---

Introduction

In this project, we implemented Federated Averaging for temporal human-activity recognition using wearable sensor data from the Localization Data for Person Activity Dataset from UCI.

We evaluated how the number of clients, local epochs, mini-batch size, and learning rate affect convergence and final model utility. We then evaluated both the aggregated global model and each client-local model after the first communication round, for membership inference using a loss-based attack. Finally, we implemented an additional privacy enhanced configuration combining per-example gradient clipping and Gaussian noise with CKKS homomorphic encrypted aggregation under an honest-but-curious server threat model.

Dataset

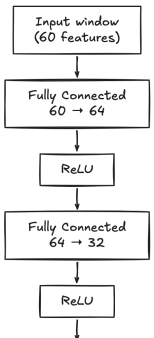
Each row contains a timestamped observation from one RF tag worn on the left ankle, right ankle, belt, or chest, including its 3D room position (x,y,z), an activity label, and an identifier for the continuous recording it belongs to.

We take the top 5 activities, synchronize the four body-worn tags into a single timestep with 12 spatial features, then group five consecutive timesteps into a non-overlapping window containing 60 features.

Activity	**Total Samples**	**50% for train/test**
Lying	2,289	1,144
Walking	1,290	645
Sitting	1,133	567
Standing up from lying	733	367
Sitting on the ground	535	267
Total	**5,980**	**2,990**

Implement FedAvg (Task A)

For each client-count experiment, the training set was partitioned into approximately equal, class-stratified client subsets to simulate IID federated clients. All other hyperparameters were held at the baseline values while one factor was varied at a time.

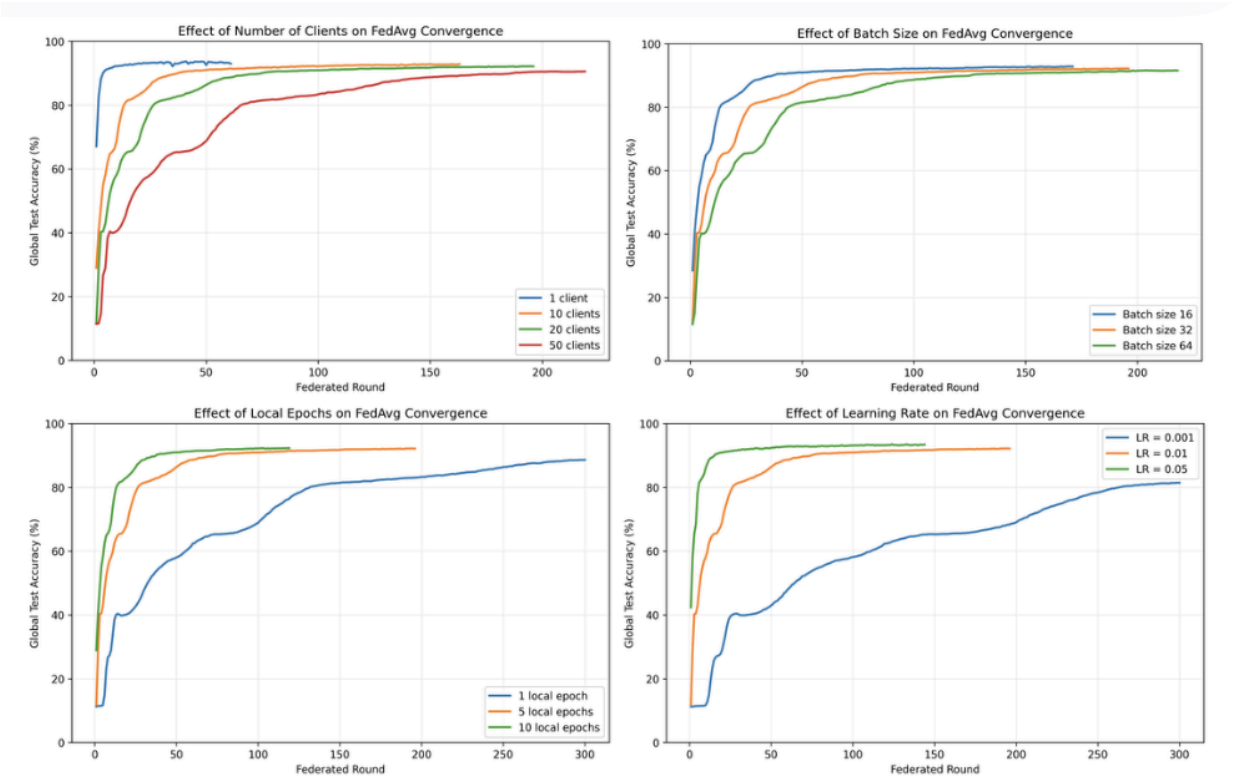


We use a lightweight multilayer perceptron with architecture 60 → 64 → 32 → 5, with ReLU activations between the hidden layers. The model contains 6,149 trainable parameters and uses no BatchNorm, dropout, momentum, or weight decay.

Training uses SGD with cross-entropy loss. The global model is trained for at least 30 federated rounds. After each round, we record test accuracy and track the best accuracy observed so far. A round is considered an improvement only if accuracy increases by at least 0.001. After the minimum 30 rounds, training stops when there are 20 consecutive rounds without such an improvement, with a maximum of 300 rounds as a computational limit.

The following configurations were studied and the results were observed:

#	Clients	Local Epochs	Batch Size	Learning Rate	Stop Round	Best Test Accuracy	Final Macro F1
1	1	5	32	0.01	61	93.75%	0.914
2	10	5	32	0.01	163	92.81%	0.908
3	20	5	32	0.01	196	92.11%	0.898
4	50	5	32	0.01	219	90.50%	0.873
5	20	1	32	0.01	300*	88.56%	0.840
6	20	10	32	0.01	119	92.21%	0.901
7	20	5	16	0.01	171	92.78%	0.910
8	20	5	64	0.01	218	91.57%	0.889
9	20	5	32	0.001	300*	81.37%	0.723
10	20	5	32	0.05	144	93.48%	0.918



Effect of Federated Learning Configuration

The experiments show that the federated configuration affects both convergence speed and final model performance.

Number of clients
Increasing the number of clients slowed convergence and slightly reduced accuracy. The 1-client configuration achieved **93.75%** best test accuracy and converged in 61 rounds, while the 50-client configuration achieved **90.50%** and required 219 rounds. Since the overall training set is fixed, increasing the number of clients reduces the amount of local data available to each client before aggregation.

Likely reason: With more clients, each client gets fewer training samples. This means each client does less learning before the models are averaged, which can slow training and lead to a slightly lower final accuracy.

Local epochs
Increasing local epochs reduced the number of communication rounds required. With 10 local epochs, the model converged in **119 rounds** and achieved **92.21%** accuracy, compared with 196 rounds and **92.11%** for the 5-epoch baseline. With only one local epoch, the model did not satisfy the convergence criterion within 300 rounds.

Likely reason: More local epochs let each client train for longer before sending its model back to the server. This means more learning happens in each communication round, so fewer rounds are needed.

Batch size
Smaller batches improved convergence in our experiments. Batch size 16 achieved **92.78%** accuracy and converged in 171 rounds, while batch size 64 achieved **91.57%** and required 218 rounds.

Likely reason: Smaller batches create more training steps within each epoch. This gives the model more chances to update its weights before aggregation, which likely helped it converge faster.

Learning rate
Learning rate produced the largest optimization difference. A learning rate of **0.001** reached only **81.37%** accuracy before the 300-round limit, whereas **0.05** achieved **93.48%** accuracy with the highest Macro F1 of **0.918**.

Likely reason: A very small learning rate makes only small changes to the model weights, so training progresses slowly. A learning rate of 0.05 made larger updates and still remained stable, so it converged faster and reached better performance.

Threat Model

We assume an **honest-but-curious server** that follows the FL protocol but may try to infer information from client updates. CKKS hides individual plaintext updates, while DP-style clipping and noise reduce the influence of individual training samples. We do not consider malicious clients, poisoning, collusion, or side-channel attacks.

Membership Inference Attack (Task B)

To evaluate whether the federated models reveal information about their training data, we perform a loss-based membership inference attack (LOSS MIA) after the first communication round. For each sample, we compute its cross-entropy loss under the model. Samples with lower loss are treated as more likely to have been part of the model's training data.

For the global model, all federated training samples are treated as members and the held-out test samples as non-members. For each client-local model, that client's own training samples are treated as members, while samples belonging to the other clients and the test set are treated as non-members.

We evaluate attack performance using Balanced Accuracy, ROC AUC, TPR at 1% FPR, TPR at 5% FPR, TPR at 10% FPR

#	Global BA	Global AUC	Mean Local BA	Mean Local AUC	Global TPR @ 1/5/10% FPR	Mean Local TPR @ 1/5/10% FPR
1 (Clients = 1)	0.506	0.499	0.506	0.499	0.007 / 0.049 / 0.095	0.007 / 0.049 / 0.095
2 (Clients = 10)	0.504	0.500	0.520	0.507	0.009 / 0.048 / 0.101	0.009 / 0.050 / 0.104
3 (Clients = 20)	0.506	0.500	0.529	0.510	0.009 / 0.054 / 0.099	0.012 / 0.058 / 0.103
4 (Clients = 50)	0.506	0.500	0.542	0.512	0.010 / 0.053 / 0.095	0.011 / 0.058 / 0.101
5 (Epochs = 1)	0.505	0.500	0.523	0.502	0.011 / 0.053 / 0.096	0.010 / 0.054 / 0.097
6 (Epochs = 10)	0.504	0.500	0.530	0.513	0.009 / 0.049 / 0.100	0.011 / 0.057 / 0.108
7 (Batch = 16)	0.505	0.500	0.529	0.513	0.009 / 0.049 / 0.101	0.012 / 0.055 / 0.110

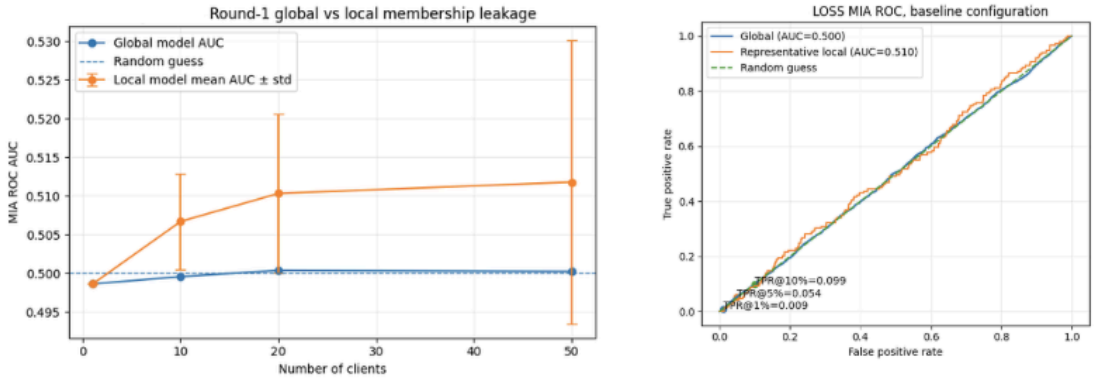
8 (Batch = 64)	0.505	0.500	0.526	0.506	0.009 / 0.054 / 0.097	0.012 / 0.057 / 0.099
9 (LR = 0.001)	0.504	0.500	0.521	0.501	0.011 / 0.054 / 0.096	0.011 / 0.052 / 0.097
10 (LR = 0.05)	0.504	0.499	0.530	0.514	0.006 / 0.049 / 0.101	0.010 / 0.054 / 0.107

The global models remain very close to AUC = 0.50 for every configuration, meaning the LOSS attack performs approximately like random guessing against the aggregated model. At 5% FPR the global TPR is also roughly 5%.

The local models show slightly more membership leakage. For a 20-client configuration, global AUC is 0.500, while mean local AUC is 0.510.

A likely reason is that each local model is trained directly on one client's small dataset and can fit those samples more closely. When the server averages models from many clients, these client-specific effects are reduced.

More aggressive local training also shows slightly higher leakage. Increasing local epochs from 1 to 10 raises mean local AUC from 0.502 to 0.513, while increasing the learning rate from 0.001 to 0.05 raises it from 0.501 to 0.514. Smaller batches also show slightly higher leakage than larger batches. However, all of these AUC values remain close to 0.50, so the observed membership signal is weak.



One likely reason for the near-random global MIA performance is the structure of the dataset and train/test split. Each sample is only a short five-timestep window, and many windows from the same activity can have very similar spatial patterns. In addition, the random window-level split means that different windows from the same recording or person can appear in both training and test sets. As a result, a non-member window may look very similar to a member window, causing their loss distributions to overlap and making exact membership difficult to distinguish. FedAvg aggregation further reduces client-specific effects in the global model.

Discussion: Future of FL (Task C)

1. There are several practical limitations to applying federated learning in real-world systems. Because training happens directly on client devices, differences in hardware, compute and memory capacity, network bandwidth, and client availability can all affect training. The set of devices participating in any given round may also not be truly random, since participation depends on factors such as connectivity, charging state, and device availability, which can introduce bias into the global model. In addition, as our experiments show, stronger privacy mechanisms can come with significant tradeoffs in model utility, communication cost, computation, and latency.

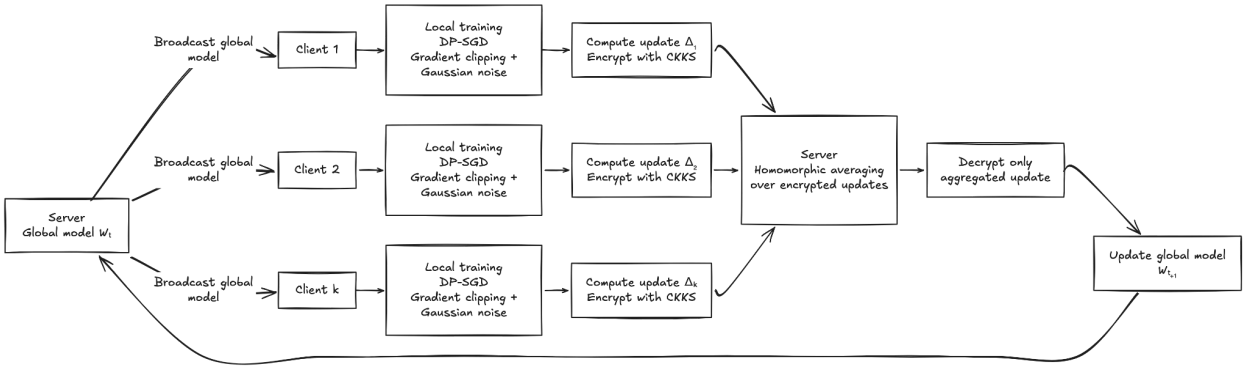
2. If better techniques are developed to reduce the communication, computation, and deployment bottlenecks of federated learning, its real-world adoption could increase. The main appeal of FL is the ability to benefit from client data without directly collecting the raw data. However, keeping the data on-device does not completely prevent information leakage, since model or gradient updates can sometimes be used to reconstruct information about the underlying training data. Wider adoption will therefore also depend on stronger and more practical privacy mechanisms that protect sensitive information at rest, in transit, and during computation, without introducing excessive utility or system overhead. Stronger privacy legislation and restrictions on sharing sensitive data could also create greater incentives for organizations to adopt FL. Finally, more mature and easy-to-use frameworks for privacy-enhanced federated learning could lower the engineering barrier and make both research and real-world deployment more accessible.

3. Our wearable activity-recognition task is a natural use case for federated learning because the data is generated directly on personal devices, is privacy-sensitive, and is naturally partitioned across users. The FL survey identifies these characteristics, particularly decentralized data, non-IID client distributions, and privacy concerns, as central motivations for cross-device federated learning. This type of wearable sensor data is also favorable for our use case because individual training samples are short, windowed numerical signals with substantial similarity across examples. In our experiments, this made exact membership difficult to infer using the LOSS attack, with the global model remaining close to random guessing. Combined with the fact that the data is naturally generated and stored on personal devices, this makes federated learning a good fit for the application.

4. A possible alternative to federated learning for our use case is fully on-device personalized learning. Since wearable activity data is naturally generated on the user's device, each device could train or fine-tune its own model without sharing updates with a central server. This would further reduce privacy and communication concerns and could better adapt to individual movement patterns. However, it would lose the main benefit of FL: learning shared patterns across many users. Therefore, this approach would be preferable only if personalization is more important than building a strong global model.

Privacy-Enhanced FL with DP + CKKS

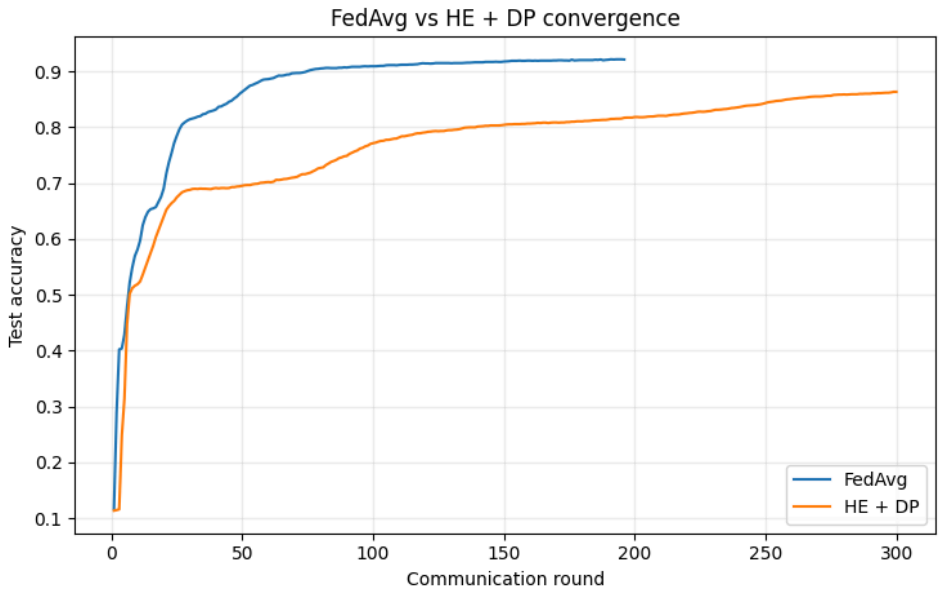
Since the LOSS attack was already close to random guessing, we used a privacy-enhanced configuration to study the tradeoffs of adding stronger privacy protection. We combined differentially private local training with CKKS homomorphic encrypted aggregation and compared the effects on utility, membership leakage, and system overhead.



We use the baseline configuration of 20 clients, 5 local epochs, batch size 32, and learning rate 0.01. Local training uses per-example gradient clipping with $C = 1.0$ and Gaussian noise with noise multiplier $\sigma = 0.5$. Client updates

are encrypted using CKKS before being sent to the server, allowing the server to average the updates without directly seeing each client's plaintext update.

Method	**Test Accuracy**	**Macro F1**	**Global Balanced Acc.**	**Global ROC AUC**	**TPR @ 1% FPR**	**TPR @ 5% FPR**	**TPR @ 10% FPR**	**Mean Local ROC AUC**
FedAvg	92.1%	0.898	0.506	0.5004	0.009	0.054	0.099	0.5103
HE + DP	86.35%	0.780	0.505	0.4999	0.009	0.052	0.096	0.5048



The privacy-enhanced configuration slightly reduced local membership leakage, with mean local ROC AUC decreasing from **0.5103 to 0.5048**, while global ROC AUC remained essentially unchanged near random guessing. However, this came with a noticeable utility cost: test accuracy decreased from **92.1% to 86.35%**, and Macro F1 decreased from **0.898 to 0.780**. This shows that adding stronger privacy protection can reduce client-level membership signal, but it also introduces a clear privacy–utility tradeoff.

CKKS also increases system cost. Encrypted communication was approximately **12.64 MiB per round**, compared with about **0.47 MiB per round** for plaintext FedAvg, roughly a **27× increase**. Encryption took about **0.287 seconds per round**, while encrypted aggregation and decryption added smaller additional overhead.